# PS12 - Bayesian Network for Traffic Monitoring

AIMLCZG557 / AECLZG557 · Assignment 2 · Weightage 13% · Group 108


| Reg. No. | Name |
|---|---|
| 2025AA05041 | Amiya Palai |
| 2025AB05180 | Arthika G |
| 2025AB05206 | Srineveda R S |
| 2025AB05203 | Aswathy H |
| 2025AB05229 | Kutarekar Nishcha Ajay |


Problem context: a smart traffic monitoring system must estimate the likelihood of road accidents and traffic signal failures from noisy, partial sensor evidence. We model both situations as common-cause Bayesian Networks and answer every query with a single reusable exact inference engine based on joint enumeration.

## 1. Task A - PEAS Description

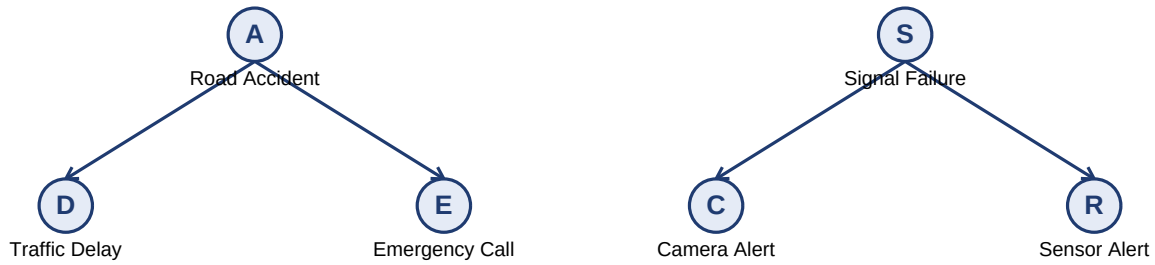
The agent is a partially observable, stochastic, sequential monitoring system for an urban / highway road network. Its role is to fuse noisy sensor evidence into calibrated risk estimates and route those estimates to human operators and downstream actuators.

| | |
|---|---|
| **Performance measure** | Calibrated posterior probability of accident and signal failure; high recall on genuine incidents; low false-alert rate; short belief-update latency; ranked risk that is actually useful to the operator on shift. |
| **Environment** | Urban and highway segments where the true state (accident, signal failure) is hidden. Observable symptoms - delay, emergency call, camera and sensor alerts - are noisy and may overlap. Weather and congestion act as unobserved influences; the environment is partially observable, stochastic, dynamic and sequential. |
| **Actuators** | Emit accident / signal-failure alerts to the traffic control room; escalate emergency-call prioritisation; request signal inspection or traffic diversion; publish structured posteriors for downstream analytics and audit logs. |
| **Sensors** | Traffic-delay detectors on road segments (D); emergency-call feeds (E); roadside camera alert modules (C); signal-health sensors (R); and, if the network is extended later, weather and congestion feeds. |

## 2. Task B - Bayesian Network Implementation

### 2.1 Network structures

Both scenarios share the same shape: one hidden cause node with two conditionally-independent effect nodes. Because the two effects have no direct edge between them, they are d-separated by the cause: given the cause, one effect gives no extra information about the other. Without conditioning on the cause, however, they are marginally dependent because they share it.




**Scenario 1 factorization**: P(A, D, E) = P(A) × P(D | A) × P(E | A).
**Scenario 2 factorization**: P(S, C, R) = P(S) × P(C | S) × P(R | S).

All variables are binary with values T and F, so each joint distribution has $2^3 = 8$ atoms.

### 2.2 Joint probability table (Scenario 1, sample input)

| A | D | E | P(A, D, E) |
|---|---|---|---|
| T | T | T | 0.0505 |

| | | | |
|---|---|---|---|
| T | T | F | 0.0111 |
| T | F | T | 0.0069 |
| T | F | F | 0.0015 |
| F | T | T | 0.0233 |
| F | T | F | 0.2093 |
| F | F | T | 0.0698 |
| F | F | F | 0.6278 |

Rows sum to 1.0 (verified in code as a sanity check). The high mass on (F, F, F) - about 0.63 - reflects that most of the time no accident, no delay and no emergency call is observed.

### 2.3 Reusable inference by joint enumeration

A single function **enumerate_prob(joint, query, value, evidence)** answers every conditional query. It sums joint atoms consistent with the evidence and normalises. Independence checks re-use the same marginalisation:

$P(Q = q \mid e) = \Sigma_{\text{rows s.t. } Q=q \wedge e} P(\text{atom}) \;/\; \Sigma_{\text{rows s.t. } e} P(\text{atom})$

Worked example: P(A | D) = P(A, D) / P(D) = (0.0505 + 0.0111) / (0.0505 + 0.0111 + 0.0233 + 0.2093) ≈ 0.0616 / 0.2942 = **0.2095**. The same routine, unchanged, computes P(A | E), P(A | D, E), P(S | C), P(S | R) and P(S | C, R) - no per-query formulas are hard-coded.

### 2.4 Results on the sample input

| Query | Posterior | Interpretation |
|---|---|---|
| P(A \| D) | 0.2095 | One symptom (delay) triples the prior 0.07. |
| P(A \| E) | 0.3816 | Emergency call is a stronger signal than delay. |
| P(A \| D and E) | 0.6848 | Two consistent symptoms almost make it likely. |
| P(S \| C) | 0.2830 | Camera alert alone: moderate belief. |
| P(S \| R) | 0.3640 | Sensor alert alone: slightly stronger. |
| P(S \| C and R) | 0.8111 | Both channels firing: near-certain failure. |

### 2.5 Independence analysis

Marginal test compares P(D=T, E=T) with P(D=T) P(E=T). Numerically 0.0738 ≠ 0.2942 × 0.1505 = 0.0443, so **D and E are not marginally independent** - as expected because they share the hidden cause A. Conditional test compares P(D=T, E=T | A) with P(D=T | A) P(E=T | A) for A ∈ {T, F}: both hold, so **D and E are conditionally independent given A**. The same verdict holds for (C, R) given S in Scenario 2. This matches the graph structure via d-separation.

## 3. Task C - Bayesian Network vs Rule-based Reasoning

**Probabilistic vs IF-THEN.** A rule engine fires when a crisp antecedent is true - e.g. IF delay AND emergency-call THEN accident. A Bayesian Network instead maintains a graded belief that is useful even when only one symptom is observed or when signals partially contradict each other. That difference is exactly what a traffic operator needs at 2 AM with a single alert firing.

**Uncertain, incomplete or overlapping symptoms.** Enumeration can condition on any subset of the observable variables, so partial evidence still yields a posterior. A rule engine would need a hand-written pattern for every possible subset and cannot express confidence in a principled way.

**Conditional dependencies drive accuracy.** Treating C and R as independent effects and then multiplying naive scores would under-use the joint evidence. In our numbers, single-alert posteriors sit near 0.28 - 0.38 but the joint (C, R) posterior jumps to 0.81. A fixed rule set cannot smoothly represent this escalation without an explosion of tuned thresholds.

**Evidence propagation.** Each new observation re-normalises the joint distribution, so the posterior moves continuously as evidence arrives. The operator sees belief grow or shrink, not a boolean flag flipping.

**Conditional independence and d-separation reduce complexity.** Because effects are d-separated by the cause, each scenario needs only three CPTs and a $2^3$ = 8 row joint instead of an unstructured full table. On a bigger road-network model with n variables, exploiting d-separation is what makes exact inference tractable at all - it prunes summations that would otherwise be exponential.

## 4. Alternate Modelling Approach

**Alternative:** a prioritised deterministic rule-based expert system over the same sensors, optionally decorated with hand-tuned confidence scores. This is trivial to build and fast, but it does not represent uncertainty in a principled way, and every new sensor forces a rewrite of many overlapping rules.

| Aspect | Bayesian Network (chosen) | Rule-based alternative |
|---|---|---|
| Uncertainty | Native posteriors from joint enumeration | Ad-hoc confidence flags |
| Partial evidence | Any subset via evidence dict | Separate rule per subset needed |
| Extending variables | Add nodes + CPTs; d-separation trims cost | Rule explosion, brittle interactions |
| Runtime (3 binary vars) | 8-row enumeration; effectively instant | Also instant |
| Scaling to city network | Prefer variable elimination or junction tree | Rule base becomes unmaintainable |

**Performance implication.** At this size both approaches are instantaneous. As the model grows to weather + congestion + many road segments, naive enumeration becomes $O(2^n)$ and would be replaced by variable elimination or a junction tree, while the rule alternative grows combinatorially and drifts out of sync with reality. The BN approach therefore scales structurally where the rule engine scales only in operator effort.

## 5. Implementation Notes

The submission is a single module **assignment.py**. Key pieces:

| Component | Role |
|---|---|
| parse_input | Reads inputPS12.txt. Rejects unknown problem id, missing / duplicate / unknown keys, unknown scenario headers, out-of-range probabilities and non-numeric values, each with a clear message. |
| JointProbabilityTable | Bounded (capacity 8) store for joint atoms. insert / delete raise ValueError when the table is full or empty, satisfying the "insert/delete must throw appropriate messages" instruction. |
| CommonCauseBN | Holds the five parameters {P(X), P(Y\|X), P(Y\|~X), P(Z\|X), P(Z\|~X)} and materialises the 8-row joint in T-before-F order. |
| enumerate_prob / marginal | One reusable exact-inference routine; no per-query formulas. |
| are_marginally_independent<br>are_conditionally_independent | Numerical independence checks with a 1e-9 tolerance. |
| run_scenario_1 /<br>run_scenario_2<br>generate_output | Format Scenario 1 (joint + queries + verdicts) and Scenario 2 (queries + verdicts) exactly per the sample output. |
| main | Reads inputPS12.txt next to the script, writes outputPS12.txt in the same folder and exits non-zero on parse errors. |

**Running it:** from the submission folder, `python assignment.py`. The program reads `inputPS12.txt` and writes `outputPS12.txt`. No values are hard-coded: swapping the input file changes the output.

**Reference:** Russell & Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., chapters on Bayesian networks and exact inference by enumeration.